Interview Question Bank (With Answers)

Candidate Profile: B.Com background, working hands-on with Alteryx, Python, ETL, and data analysis (business/finance data focus, not a pure engineering background)

1. Alteryx

1. What is Alteryx and what kind of business problems have you used it to solve in your role?

Answer: Alteryx is a drag-and-drop data blending and automation tool used to combine, clean, and transform data from multiple sources without heavy coding. It's commonly used to automate recurring reports, merge Excel/CSV/database data, and build reconciliation or MI workflows that previously took hours to do manually.

2. Walk me through a workflow you've built in Alteryx — what were the input files, what transformations did you apply, and what was the output?

Answer: A good answer describes: input files (e.g., two Excel exports), cleaning steps (removing blanks, standardizing formats), a Join or Union to combine them, a Filter/Formula step to flag exceptions, and an output (Excel report or database table). Look for a clear, logical flow from input to output.

3. What is the difference between the Join tool and the Union tool? When would you use each?

Answer: Join combines two inputs side-by-side based on a matching key (like a SQL join), producing matched, left-only, and right-only outputs. Union stacks two inputs with the same columns on top of each other (row-wise combination), used when appending similar datasets like monthly files.

4. How do you use the Filter tool and Formula tool together to clean or categorize data?

Answer: The Formula tool creates or modifies a column (e.g., flag a row as 'Break' if amounts don't match), and the Filter tool then splits records into True/False branches based on a condition, letting you route breaks and matches down different paths of the workflow.

5. What is the Fuzzy Match tool used for, and have you used it to match records that don't have identical keys (e.g., slightly different names)?

Answer: Fuzzy Match finds records that are similar but not identical — useful when names or descriptions have typos, abbreviations, or formatting differences (e.g., 'ABC Ltd' vs 'ABC Limited'). It uses similarity scoring to suggest likely matches instead of requiring an exact key match.

6. How do you combine data from multiple Excel/CSV files that have the same structure using Alteryx?

Answer: Use a Directory tool or wildcard input to pick up all files in a folder, then feed them into a Union tool (or use a batch macro) to stack them into a single combined dataset, provided all files share the same column structure.

7. What is the Summarize tool used for? Give an example of a business report you built using it.

Answer: Summarize performs aggregations like sum, count, average, or group-by, similar to a pivot table. For example, summarizing total transaction amount by department or region to build a monthly summary report for management.

8. How do you handle blank/missing values in a dataset within Alteryx?

Answer: Common approaches include the Data Cleansing tool to replace nulls with a default value, or a Filter tool to separate and review records with missing key fields before they flow further into the workflow.

9. What is a Macro in Alteryx, and have you built or used one to make a workflow reusable?

Answer: A macro is a reusable mini-workflow that can be plugged into a larger workflow, useful when the same logic (e.g., a standard cleaning step or matching rule) needs to be applied across multiple projects or datasets without rebuilding it each time.

10. How do you schedule an Alteryx workflow to run automatically, and how do you know if it failed?

Answer: Workflows can be scheduled via Alteryx Server (or Windows Task Scheduler for Designer) to run at a set time. Failures are typically flagged through email notifications, run logs, or a Message tool configured to alert if row counts or conditions look wrong.

11. How would you build a workflow to compare two files (e.g., last month vs this month) and highlight the differences?

Answer: Join the two files on a common key, then use a Formula tool to compare values field-by-field and flag mismatches, additions, or deletions. Output separate tabs or files for 'new records', 'removed records', and 'changed records'.

12. What steps would you take if your Alteryx workflow output has an unexpected number of rows compared to the input?

Answer: Check for duplicate keys causing a many-to-many join (row multiplication), review Filter conditions that may be dropping valid rows, and use a Browse tool at each step to trace exactly where the row count changes.

13. Have you connected Alteryx to a database or used the In-DB tools? Describe the use case.

Answer: In-DB tools let Alteryx push processing down to the database instead of pulling all data into memory first, which is useful for very large datasets. A common use case is filtering and aggregating millions of rows in a SQL database before bringing only the summarized result into Alteryx.

14. How do you document a workflow so someone else on your team can understand and maintain it?

Answer: Using Comment/Container tools to label logical sections of the workflow, naming tools clearly instead of leaving defaults, and maintaining a short README or process note describing inputs, outputs, and the business purpose.

15. Describe a time an Alteryx workflow saved significant manual effort compared to doing the task in Excel.

Answer: Look for a concrete example — e.g., a monthly reconciliation that took several hours in Excel with manual VLOOKUPs was reduced to a few minutes by automating the join, cleaning, and summary steps in Alteryx, with fewer manual errors.

2. ETL

1. In simple terms, what does ETL mean, and where does it fit in a typical reporting or reconciliation process?

Answer: ETL stands for Extract, Transform, Load — extracting data from source systems, transforming/cleaning it into the required format, and loading it into a target system or report. It sits between raw source data and the final report or reconciliation output.

2. Describe an ETL process you've worked on — what was the source data, what transformations were applied, and where did it land?

Answer: Look for a clear description: source (e.g., a daily system extract), transformations (cleaning, standardizing formats, joining reference data), and target (a database table, Excel report, or dashboard).

3. What is the difference between a full load and an incremental load? Which have you worked with?

Answer: A full load reloads the entire dataset each time, simple but slower for large data. An incremental load only picks up new or changed records since the last run, which is faster and more efficient for daily processes.

4. How do you check that data hasn't been lost or duplicated after loading it from source to target?

Answer: Compare row counts and key totals (e.g., sum of amount column) between source and target, and spot-check a sample of records to confirm values match exactly.

5. What checks would you do on a file before using it — e.g., checking row counts, missing columns, or blank values?

Answer: Verify the file has the expected number of columns and headers, check for unexpected blank or null values in key fields, confirm row counts are in a reasonable range compared to prior days, and check for duplicate keys.

6. How do you handle a source file that arrives late or in the wrong format?

Answer: Build in a check or wait step before processing starts, and have a fallback alert/notification if the file hasn't arrived by a set time. If the format is wrong, flag it for manual review rather than letting the workflow process bad data silently.

7. What tools have you used to move/schedule data pipelines (Alteryx Server, Task Scheduler, Airflow, or others)?

Answer: Candidates should describe the specific scheduling tool they've used and how they monitor for failures — e.g., Alteryx Server schedules with email alerts on failure, or Windows Task Scheduler running a script daily.

8. How would you combine data from two different systems (e.g., an Excel export and a database extract) into one report?

Answer: Standardize both datasets to common column names and formats first, then join or merge them on a shared key (e.g., an ID or account number) before summarizing into the final report.

9. What would you do if an ETL job runs successfully but the output numbers look wrong?

Answer: Trace back step-by-step from the output to the source, checking each transformation stage for where the numbers diverge — often caused by a bad join, incorrect filter, or a data type/formatting issue (e.g., text vs number).

10. How do you keep track of different versions of a workflow or script as it evolves over time?

Answer: Save dated or versioned copies of workflows/scripts, use a shared drive or version control (like Git for scripts), and keep a simple change log describing what was updated and why.

11. Describe how you validate that a monthly or daily automated report matches what was previously done manually.

Answer: Run the automated process in parallel with the manual process for a period, compare totals and key figures line-by-line, and only fully switch over once the two consistently match.

12. What is data cleaning, and what are some common cleaning steps you apply before analysis (trimming spaces, standardizing formats, removing duplicates)?

Answer: Data cleaning is preparing raw data for accurate analysis — removing extra spaces, standardizing date/text formats, correcting inconsistent naming, removing duplicate records, and handling missing values before the data is used further.

3. Python

1. What Python libraries have you used for working with data, and what is pandas used for?

Answer: Pandas is the main library for working with tabular data — reading files, filtering, grouping, merging, and exporting results. Other commonly mentioned libraries include openpyxl (Excel), numpy (numeric operations), and matplotlib (charts).

2. How do you read an Excel or CSV file into Python and view the first few rows?

Answer: import pandas as pd; df = pd.read_csv('file.csv') or pd.read_excel('file.xlsx'), then df.head() to preview the first 5 rows.

3. How would you filter a dataframe to show only rows where a column value meets a certain condition (e.g., amount > 1000)?

Answer: df[df['amount'] > 1000] — this uses a boolean condition inside square brackets to return only matching rows.

4. How do you find and remove duplicate rows in a dataset using pandas?

Answer: df.duplicated().sum() to count duplicates, and df.drop_duplicates() to remove them, optionally specifying a subset of columns to check.

5. How do you handle missing or blank values in a column — for example, filling them with 0 or dropping those rows?

Answer: df['col'].fillna(0) to fill missing values with a default, or df.dropna(subset=['col']) to remove rows where that column is blank.

6. What is the difference between merge() and concat() in pandas, and when would you use each?

Answer: merge() joins two dataframes side-by-side based on a common key (like a SQL join), while concat() stacks dataframes on top of each other (or side-by-side) without needing a matching key — used for combining similar files.

7. How would you group data by a column and calculate totals or averages (e.g., total amount by department)?

Answer: df.groupby('department')['amount'].sum() for totals, or .mean() for averages, then .reset_index() to turn it back into a clean dataframe.

8. How do you write a dataframe back to an Excel or CSV file?

Answer: df.to_excel('output.xlsx', index=False) or df.to_csv('output.csv', index=False).

9. What is a for loop, and can you give an example of using one to process a list of files?

Answer: A for loop repeats an action for each item in a list — for example, looping through a list of filenames and reading/processing each one in turn, often combining the results afterward.

10. How do you write a simple function in Python, and why is it useful to break code into functions?

Answer: def my_function(param): ... return result. Functions make code reusable, easier to test, and easier to read since related logic is grouped and named instead of repeated inline.

11. How do you use try/except to handle an error, such as a file that doesn't exist?

Answer: Wrap the risky code in a try block and catch the specific error in an except block (e.g., except FileNotFoundError), so the program can print a message or continue instead of crashing.

12. Have you used Python to automate a repetitive task you used to do manually in Excel? Describe it.

Answer: Look for a concrete example — e.g., automating a daily file merge and summary that used to involve manual copy-pasting and VLOOKUPs, reducing the task from an hour to a few seconds.

13. How would you compare two lists (or columns) to find values that exist in one but not the other?

Answer: Using set operations: set(list1) - set(list2) gives values in list1 not in list2. For dataframe columns, df1[~df1['col'].isin(df2['col'])] achieves the same.

14. What is a dictionary in Python and when have you used one (e.g., to map codes to names)?

Answer: A dictionary stores key-value pairs, e.g., {'US': 'United States', 'UK': 'United Kingdom'} — useful for quick lookups, such as mapping short codes to full names when cleaning data.

4. Data Analysis

1. Walk me through how you approach analyzing a new dataset you've never seen before.

Answer: Start by understanding the structure (columns, row count, data types), check for missing or inconsistent values, look at summary statistics, and form an initial view of trends before diving into deeper analysis or specific questions.

2. What steps do you take to make sure the data you're analyzing is accurate and complete before drawing conclusions?

Answer: Cross-check totals against a source system or prior report, look for unexpected blanks or duplicates, and sanity-check a few individual records manually before trusting the aggregated numbers.

3. How would you identify outliers or unusual values in a dataset, and what would you do with them?

Answer: Look for values far outside the normal range (e.g., using a simple threshold, sorting, or a boxplot), then investigate whether they're genuine (a large but valid transaction) or an error (a data entry mistake) before deciding to exclude or flag them.

4. What is a pivot table, and describe a business question you've answered using one.

Answer: A pivot table summarizes data by grouping and aggregating it (e.g., sum, count, average) across categories — for example, using one to show total spend by region and month to identify which region grew fastest.

5. How do you decide which chart type to use to present a given set of data (bar, line, pie)?

Answer: Use a line chart for trends over time, a bar chart for comparing categories, and a pie chart sparingly for showing proportion of a whole (and only with a small number of categories).

6. How would you summarize month-on-month or year-on-year trends for a stakeholder who isn't technical?

Answer: Focus on the headline number and direction of change (e.g., 'up 12% vs last month'), use a simple trend chart, and avoid jargon — explain the 'so what' rather than just showing raw numbers.

7. What is the difference between correlation and causation, and why does it matter when presenting analysis?

Answer: Correlation means two things move together; causation means one directly causes the other. It matters because presenting a correlation as if it proves causation can mislead stakeholders into making incorrect decisions.

8. How would you investigate why a number in your report suddenly changed a lot compared to last month?

Answer: Break the total down into its components (by category, region, or source) to isolate where the change came from, then check if it's a genuine business change or a data/process issue (e.g., a missing file or a new source field).

9. Describe a report or dashboard you've built and what business decisions it supported.

Answer: Look for a clear example connecting the report to a real decision — e.g., a break-trend report that helped management decide to add headcount to a high-break-volume area.

10. How do you validate that your analysis/report numbers tie back to the original source data?

Answer: Reconcile key totals in the report against the source system directly, and periodically spot-check individual transactions to confirm the report logic hasn't introduced errors.

11. What tools have you used to build dashboards or reports (Excel, Power BI, Tableau, etc.)?

Answer: Candidates should name the specific tools they've used and briefly describe what they built — e.g., a Power BI dashboard refreshed daily from an Alteryx output.

12. How would you present the same set of findings differently to a team member versus a senior manager?

Answer: With a team member, go into more operational detail and methodology; with a senior manager, lead with the headline conclusion and business impact, keeping supporting detail available but not central.

5. SQL

1. What is the difference between INNER JOIN and LEFT JOIN? Give a simple example.

Answer: INNER JOIN returns only rows that have a match in both tables. LEFT JOIN returns all rows from the left table plus matching rows from the right table (with NULLs where there's no match) — e.g., all customers, whether or not they have an order.

2. Write a query to find duplicate values in a column.

Answer: SELECT trade_id, COUNT(*) FROM trades GROUP BY trade_id HAVING COUNT(*) > 1;

3. Write a query to find records in one table that don't have a matching record in another table.

Answer: SELECT a.* FROM table_a a LEFT JOIN table_b b ON a.id = b.id WHERE b.id IS NULL;

4. How would you calculate the total and average of a numeric column, grouped by category?

Answer: SELECT category, SUM(amount) AS total, AVG(amount) AS average FROM transactions GROUP BY category;

5. What is the difference between WHERE and HAVING?

Answer: WHERE filters individual rows before any grouping happens; HAVING filters groups after a GROUP BY/aggregation has been applied (e.g., HAVING COUNT(*) > 1).

6. How would you find the top 5 highest values in a table?

Answer: SELECT * FROM transactions ORDER BY amount DESC LIMIT 5;

7. What does GROUP BY do, and when do you need to use it?

Answer: GROUP BY groups rows that share the same value in specified columns so aggregate functions (SUM, COUNT, AVG) can be applied per group, e.g., total sales per region.

8. How would you write a query to count how many records fall into different date ranges (e.g., last 7 days, last 30 days)?

Answer: SELECT CASE WHEN txn_date >= CURRENT_DATE - 7 THEN 'Last 7 days' WHEN txn_date >= CURRENT_DATE - 30 THEN 'Last 30 days' ELSE 'Older' END AS date_range, COUNT(*) FROM transactions GROUP BY date_range;

6. Python Coding Questions with Solutions

Practical, pandas-focused exercises suited to a business/data-analysis background. Give the candidate the problem only; solutions below are for the interviewer.

Q1. Read a file and view basic info

Read 'sales.csv' into a dataframe and print the number of rows, columns, and the first 5 rows.

Solution:

```
import pandas as pd

df = pd.read_csv('sales.csv')
print("Rows, Columns:", df.shape)
print(df.head())
```

Q2. Filter rows based on a condition

From a dataframe of transactions, select only the rows where 'amount' is greater than 5000.

Solution:

```
high_value = df[df['amount'] > 5000]
print(high_value)
```

Q3. Find and remove duplicate rows

Write code to find how many duplicate rows exist in a dataframe, then remove them.

Solution:

```
duplicate_count = df.duplicated().sum()
print("Duplicates found:", duplicate_count)

df_clean = df.drop_duplicates()
```

Q4. Handle missing values

Fill missing values in the 'region' column with 'Unknown', and drop rows where 'amount' is missing.

Solution:

```
df['region'] = df['region'].fillna('Unknown')
df = df.dropna(subset=['amount'])
```

Q5. Group and summarize data

Calculate the total 'amount' for each 'department'.

Solution:

```
summary = df.groupby('department')['amount'].sum().reset_index()
print(summary)
```

Q6. Compare two files and find mismatches

You have two dataframes, df1 (internal) and df2 (source system), both with a 'trade_id' and 'amount' column. Find trade_ids where the amount differs between the two.

Solution:

```
merged = df1.merge(df2, on='trade_id', suffixes=('_internal', '_source'))
mismatches = merged[merged['amount_internal'] != merged['amount_source']]
print(mismatches)
```

Q7. Find records missing from one file

Find trade_ids that exist in df1 but are missing from df2.

Solution:

```
missing = df1[~df1['trade_id'].isin(df2['trade_id'])]
print(missing)
```

Q8. Write output back to Excel

Save the 'mismatches' dataframe from Q6 to an Excel file called 'breaks.xlsx'.

Solution:

```
mismatches.to_excel('breaks.xlsx', index=False)
```

Q9. Simple function with error handling

Write a function that reads a CSV file and returns a dataframe; if the file doesn't exist, print a friendly message instead of crashing.

Solution:

```
import pandas as pd

def read_file_safely(path):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"File not found: {path}")
        return None
```

Q10. Loop through a list of files

You have a list of monthly CSV files. Write code to read each one and combine them into a single dataframe.

Solution:

```
import pandas as pd

files = ['jan.csv', 'feb.csv', 'mar.csv']
all_data = []

for f in files:
    monthly_df = pd.read_csv(f)
    all_data.append(monthly_df)

combined = pd.concat(all_data, ignore_index=True)
```